

Multi – level Linked List – Modified Insert After Element

–Time Complexity Analysis

–(Best ,Worst,Average)

Program:

```
Node *searchNode(Node *current, int target)
{
    if (current == nullptr)
        return nullptr;

    if (current->data == target)
        return current;

    Node *foundInChild = searchNode(current->child, target);
    if (foundInChild != nullptr)
    {
        return foundInChild;
    }
    return searchNode(current->next, target);
}

void insertAfterElement()
{
    if (head == nullptr)
    {
        cout << "List is empty. Please insert an element first." << endl;
        return;
    }

    int target, data, type;
    cout << "Enter the element to search for: ";
    cin >> target;

    Node *temp = searchNode(head, target);
```

```

    if (temp == nullptr)
    {
        cout << "Element " << target << " not found anywhere
in the list." << endl;
        return;
    }

    cout << "Enter data to insert: ";
    cin >> data;
    Node *newNode = createNode(data);

    cout << "Insert as (1) Next Node or (2) Child Node? ";
    cin >> type;

    if (type == 1)
    {
        newNode->next = temp->next;
        temp->next = newNode;
        cout << "Inserted " << data << " as NEXT of " <<
target << endl;
    }
    else if (type == 2)
    {
        newNode->next = temp->child;
        temp->child = newNode;
        cout << "Inserted " << data << " as CHILD of " <<
target << endl;
    }
    else
    {
        cout << "Invalid option." << endl;
        free(newNode);
    }
}
int main()
{
    int choice;

```

```

while (true)
{
    cout << "4. Insert After Element (Link or Child)" <<
endl;
    switch (choice)
    {
        case 4:
            insertAfterElement();
        default:
            cout << "Invalid choice" << endl;
    }
}
return 0;
}

```

Time Complexity Analysis:

Best Case :

The best case occurs when the target element is located at the very first node checked (the head node).

- ***Behavior:*** The function makes a single comparison $current \rightarrow data == target$, finds a match, and immediately returns the pointer without making any recursive calls.

- ***Recurrence Equation:***

$$T(N) = c \text{ (where } c \text{ is a constant time for one comparison)}$$

- ***Time Complexity:***

$$O(1)$$

Worst Case :

The worst case occurs when the target element is either not present in the list at all, or it is located at the very last node.

- ***Behavior: The function must visit every single node in the structure exactly once to exhaust all possibilities.***

1. $T(k)$: The time of the program spends searching the entire child branch.
2. $T(N - 1 - k)$: The time of the program spends searching the entire next branch.
3. c : The $O(1)$ constant time spent at the current node doing if checks.

i. e.

$$T(10) = T(6) + T(10 - 1 - 6) + c$$

$$\text{or, } T(10) = T(6) + T(3) + c$$

Every time the searchNode function runs, the total N nodes are split into exactly three distinct parts:

1. ***The current node itself:*** This is exactly 1 node.
2. ***The child branch:*** We don't know exactly how many nodes are down this branch, so we use a variable and call it k nodes.
3. ***The next branch:*** How many nodes are left for this branch?

If the total number of nodes is N , and we have already accounted

for 1 current node and k child nodes, the remaining nodes

for the next branch must be the total minus the others:

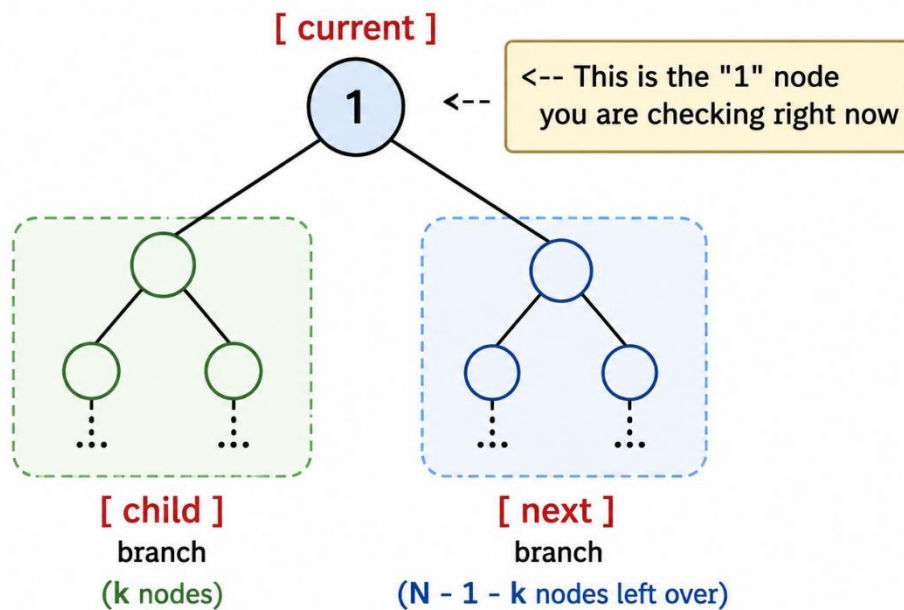
$$\text{Remaining} = N - 1 - k$$

Therefore, the time it takes to search that next branch is $T(N - 1 - k)$.

The current node refers to : $\text{current} \rightarrow \text{data} == \text{target}$. That is the current node which is being processed at a time.

$\text{Node} * \text{searchNode}(\text{Node} * \text{current}, \text{int target}) \rightarrow \text{Node} * \text{current}$ is the current node processed at a time.

$$T(N) = T(k) + T(N - 1 - k) + c$$



Since the current node itself occupies one node, the remaining

$N - 1$ nodes are divided between:

- the child branch: k nodes,
- the next branch: $N - 1 - k$ nodes.

Thus,

$$N = 1 + k + (N - 1 - k).$$

Boundary of k

The smallest possible value is when there is no child:

$$k = 0$$

The largest possible value is when all remaining nodes are in the child branch and there is no next branch:

$$k = N - 1$$

Therefore,

$$0 \leq k \leq N - 1$$

1)The smallest possible value is when there is no child:

$$T(N) = T(0) + T(N - 1 - 0) + c$$

where , $T(0) + c = c$,where `c` is constant .

$$\therefore T(N) = T(N - 1) + c$$

2)The largest possible value is when all remaining nodes are in the child branch and there is no next branch:

$$k = N - 1$$

$$T(N) = T(N - 1) + T(N - 1 - (N - 1)) + c$$

$$T(N) = T(N - 1) + T(0) + c$$

$$\text{or, } T(N) = T(N - 1) + c$$

where , $T(0) + c = c$,where `c` is constant .

3)Perfectly balanced case

$$T(N) = T(k) + T(N - 1 - k) + c, \quad 0 \leq k \leq N - 1$$

For a perfectly balanced case,

$$k = \frac{N - 1}{2}$$

$$\therefore N - 1 - k = N - 1 - \left(\frac{N - 1}{2}\right) = \frac{2N - 2 - N + 1}{2} = \left(\frac{N - 1}{2}\right)$$

$$T(N) = T\left(\frac{N - 1}{2}\right) + T\left(\frac{N - 1}{2}\right) + c$$

$$T(N) = 2T\left(\frac{N - 1}{2}\right) + c$$

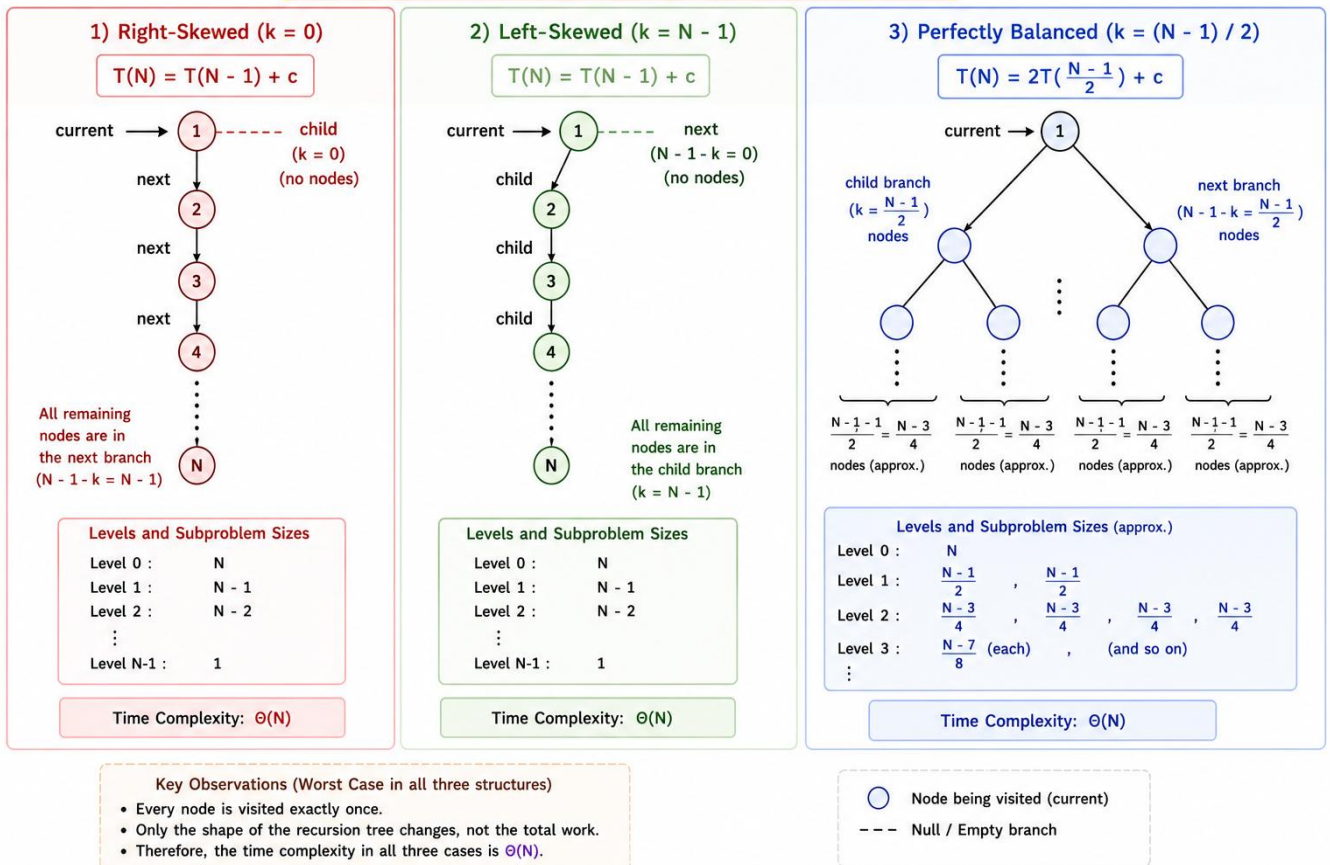
Therefore , we had three conditions :

1) *Right – Skewed* ($k = 0$) : $T(N) = T(N - 1) + c$

2) *left – Skewed* ($k = N - 1$) : $T(N) = T(N - 1) + c$

3) *Perfectly Balanced* $\left(k = \frac{N - 1}{2}\right) = 2T\left(\frac{N - 1}{2}\right) + c$

$$\text{Recurrence: } T(N) = T(k) + T(N - 1 - k) + c$$



The function visits each node exactly once, so:

$$T(N) = c \times N$$

Worst Case Complexity: $O(N)$

This three conditions will be discussed later.

3. Average Case

The average case assumes that the target element has an equal probability of being found at any node in the list.

- *Behavior: On average, the search will have to traverse half of the total nodes before finding the target.*
- *Mathematical Equation: If the target is in the list, the expected number of nodes visited is the sum of the probabilities of finding the target at position i multiplied by i steps:*

$$\begin{aligned} &\text{for position } i = 1, \text{ probability of finding the target } \frac{1}{N} \\ &\text{for position } i = 2, \text{ probability of finding the target } \frac{1}{N} \\ &\quad \cdot \\ &\quad \cdot \\ &\quad \cdot \\ &\text{for position } i = N, \text{ probability of finding the target } \frac{1}{N} \end{aligned}$$

$$\begin{aligned} E[T] &= \sum_{i=1}^N \left(\frac{1}{N} \right) \cdot (c \cdot i) + C \\ &= \frac{c}{N} \sum_{i=1}^N i + C \\ &= \frac{c}{N} \cdot \frac{N(N+1)}{2} + C \\ &= \frac{c(N+1)}{2} + C \end{aligned}$$

where,

1) *visiting one node costs a constant `c`.*

2) *C or $O(1)$ is the constant insertion cost after the target is found.*

Time Complexity: Dropping the constants, this simplifies to $O(N)$.
